SmartDB ESP32-CAM diagnostic firmware

ESP-IDF firmware for testing an AI Thinker-style ESP32-CAM, diagnosing its SCCB camera bus, capturing JPEG frames, and saving them to a microSD card.

Current status

- The firmware builds successfully.
- The current SoftAP is `esp_not_esp-ing` with password `SmartDBesp32cam`.
- The latest captured run successfully started Wi-Fi and mounted the SD card.
- Camera initialization failed because the sensor did not acknowledge SCCB reads or writes; no frames were captured in that run.

At startup the application:

1. Creates a Wi-Fi SoftAP.
2. Mounts a FAT-formatted microSD card in SDMMC 1-bit mode.
3. Checks the SCCB SDA/SCL levels and scans for a camera sensor.
4. Initializes the camera and captures 10 QVGA JPEG frames at two-second intervals.
5. Saves successful captures as `/sdcard/frame_0000.jpg` through `/sdcard/frame_0009.jpg`, then unmounts the card.

Initialization and capture results are reported over the serial monitor. An SD failure is non-fatal; a Wi-Fi initialization error may abort through `ESP_ERROR_CHECK`, and a camera initialization failure stops the test.

Hardware assumptions

The pin mapping in `main/main.c` targets the common AI Thinker ESP32-CAM camera connector:

Signal	GPIO	Signal	GPIO
XCLK	0	PCLK	22
SCCB SDA	26	SCCB SCL	27
D0-D3	5, 18, 19, 21	D4-D7	36, 39, 34, 35
VSYNC	25	HREF	23
RESET	Not connected	PWDN	Not connected

The current camera configuration uses internal DRAM rather than PSRAM, one frame buffer, QVGA (320x240), JPEG quality 12, and a 24 MHz XCLK.

The microSD card must be FAT-formatted. SDMMC and the camera share GPIOs on many ESP32-CAM boards, so concurrent use can be board-dependent. If either peripheral behaves unreliably, disable SD while testing the camera.

Build, flash, and monitor

Install ESP-IDF, connect the board through a USB-to-serial adapter, and put the ESP32 into download mode (GPIO0 held low during reset). Then run:

```
source "$HOME/esp/esp-idf/export.sh"
idf.py set-target esp32
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

Replace `/dev/ttyUSB0` with the port used by your adapter. Exit the monitor with `Ctrl+]`. After flashing, release GPIO0 and reset the board to run normally.

The repository also contains a minimal PlatformIO environment:

```
pio run
pio run --target upload
pio device monitor
```

Configuration

Runtime options are compile-time constants near the top of `main/main.c`:

Option	Default	Purpose
`ENABLE_AP`	`1`	Enable the Wi-Fi SoftAP
`AP_SSID`	`esp_not_esp-ing`	SoftAP network name
`AP_PASS`	`SmartDBesp32cam`	SoftAP password
`AP_CHANNEL`	`1`	Wi-Fi channel
`AP_MAX_CONN`	`4`	Maximum connected stations
`ENABLE_SD`	`1`	Mount and write to the microSD card

Change these values and rebuild. The SoftAP is currently for connectivity diagnostics only; the application does not start an HTTP server or stream camera images.

How to use it

1. Insert a FAT-formatted microSD card and connect the camera ribbon in the correct orientation for your module.
2. Wire the USB-to-serial adapter: TX to ESP32 RX0, RX to ESP32 TX0, and GND to GND. Power the board from a stable 5 V supply.
3. Hold GPIO0 low and reset the ESP32 to enter download mode.
4. Build and flash the firmware:

```
idf.py build
idf.py -p /dev/ttyUSB0 flash
```

5. Release GPIO0, reset the board, and open the monitor:

```
idf.py -p /dev/ttyUSB0 monitor
```

6. Watch for SoftAP startup, SD mounting, GPIO checks, the SCCB scan, camera initialization, and frame capture. Exit the monitor with `Ctrl+]`.

7. On a successful run, wait for `SD card unmounted` before removing the card. Images are stored as `/sdcard/frame_0000.jpg` through `frame_0009.jpg`.

Connect a phone or computer to `esp_not_esp-ing` using password `SmartDBesp32cam`. The ESP32 normally uses `192.168.4.1`, but the firmware does not currently provide a website or camera stream.

Save a serial session for troubleshooting with:

```
idf.py -p /dev/ttyUSB0 monitor 2>&1 | tee esp32cam-monitor.log
```

Expected serial output

A normal run includes messages similar to:

```
SmartDB ESP32-CAM basic test starting...
WiFi SoftAP started. SSID:'esp_not_esp-ing' password:'SmartDBesp32cam'
SD card mounted at /sdcard
GPIO-level check: SDA=26 SCL=27
Device responded at SCCB addr=0x30
Camera initialised successfully
Frame 0 captured:
  JPEG header: VALID
  Saved to /sdcard/frame_0000.jpg (... bytes)
Test complete — 10 frames captured
SD card unmounted
```

Exact sensor addresses, IDs, frame sizes, and memory values depend on the connected module.

Latest monitor log explained

The latest captured monitor session is `build/log/idf_py_stdout_output_514681`. It predates the current AP credentials, so its old SSID is historical output. The important results are:

- ESP32 revision 3.1 booted successfully.

- Wi-Fi started and DHCP assigned the AP address `192.168.4.1`.
- A 64 GB SDHC card mounted at `/sdcard` in 1-bit mode.
- SDA and SCL both remained high with and without internal pull-ups, so neither SCCB line appeared stuck low.
- The SCCB address scan found no responding sensor.
- Reads at address `0x30` returned `ESP_ERR_INVALID_RESPONSE`. The logged `0x28` values came from failed operations and are not trustworthy register values.
- A fallback heuristic guessed OV5640, but sensor reset and frame-size writes also failed. Camera initialization stopped before frame capture.
- The board contains 4 MB of flash, while the firmware image is configured for 2 MB. This warning is unrelated to SCCB, but the flash size can be corrected in `idf.py menuconfig` under **Serial flasher config > Flash size**.

This points to camera communication rather than Wi-Fi, SD, or framebuffer configuration. Check camera power, ribbon orientation and contacts, XCLK, SDA/SCL continuity, and the actual sensor model. Do not treat the OV5640 guess as confirmed detection.

Code explanation

All application logic is in `main/main.c`. ESP-IDF calls `app_main()` after the bootloader, scheduler, heap, and system services are ready. The functions in this file then initialize each peripheral in a fixed sequence.

Headers and SCCB declarations

The FreeRTOS headers provide `vTaskDelay()` and `pdMS_TO_TICKS()`. ESP-IDF headers provide logging, GPIO, Wi-Fi, networking, NVS, FAT filesystem, and SDMMC APIs. `esp_camera.h` supplies the camera configuration, framebuffer, and sensor types.

The four `extern` SCCB declarations expose private functions from the bundled camera component without including its private header:

- `SCCB_Init()` assigns the SDA and SCL pins and installs the SCCB/I2C bus.
- `SCCB_Probe()` checks whether a device acknowledges an address.
- `SCCB_Read16()` reads an 8-bit value from a 16-bit sensor register address.
- `SCCB_Deinit()` releases the bus after the diagnostic scan.

This direct use is useful for debugging, but it couples the application to internal camera-component APIs. If those function signatures change, this file must be updated as well.

Compile-time options and pin definitions

`ENABLE_AP` and `ENABLE_SD` decide whether their initialization blocks run. They are C preprocessor constants, so changing them requires rebuilding and flashing the firmware. The AP constants are copied into `wifi_config_t` during startup.

The `CAM_PIN_*` constants describe the physical connection between the ESP32 and camera:

- `XCLK` is the master clock generated by the ESP32 for the sensor.
- `SIOD` and `SIOC` are SCCB data and clock, similar to I2C SDA and SCL.
- `D0` through `D7` form the eight-bit parallel pixel-data bus.

- `VSYNC` marks frame boundaries, `HREF` marks valid pixels on a row, and `PCLK` clocks each byte into the ESP32.
- `PWDN` and `RESET` are `-1`, telling the driver that software cannot toggle those signals. This matters when a sensor needs a hardware reset.

`camera_config`

`camera_config_t` is passed unchanged to `esp_camera_init()`:

- `pin_*` fields connect the logical camera signals to the GPIO constants.
- `sccb_i2c_port = 1` selects I2C controller 1 for sensor register traffic.
- `xclk_freq_hz = 24000000` asks LEDC timer/channel 0 to generate a 24 MHz XCLK.
- `PIXFORMAT_JPEG` asks the sensor to produce compressed JPEG data instead of raw RGB or YUV data.
- `FRAMESIZE_QVGA` selects 320x240. This lowers memory and transfer requirements while hardware communication is being debugged.
- `jpeg_quality = 12` controls JPEG compression. In this driver, a lower number means better quality and usually a larger file.
- `fb_count = 1` allocates one framebuffer. Capture must finish using and return that buffer before another frame can be acquired.
- `CAMERA_FB_IN_DRAM` places the framebuffer in internal RAM because PSRAM is disabled in the current SDK configuration.
- `CAMERA_GRAB_WHEN_EMPTY` waits for the framebuffer to become available rather than replacing an older frame.

SD state: `sd_card` and `sd_mounted`

`sd_card` receives the card descriptor created by the mount function. `sd_mounted` starts as `false` and changes to `true` only after a successful mount. `capture_frame()` checks this flag before opening a file, and `app_main()` checks it before unmounting. This prevents file operations when no usable card exists.

`sd_init()`

The function performs these steps:

1. `SDMMC_HOST_DEFAULT()` creates the ESP32 SDMMC host configuration.
2. `SDMMC_SLOT_CONFIG_DEFAULT()` creates the slot/pin configuration.
3. `slot_config.width = 1` selects the one-bit bus used by the target board.
4. The mount configuration disables automatic formatting, allows five files to be open, and requests 16 KiB allocation units.
5. `esp_vfs_fat_sdmmc_mount()` initializes the card, mounts FAT at `/sdcard`, and connects standard C file functions such as `fopen()` to that path.
6. On success, the code prints the card information and sets `sd_mounted`.

The return value is an `esp_err_t`. `ESP_FAIL` is reported as a likely format or mount problem; other codes are printed with `esp_err_to_name()`. `app_main()` treats all SD errors as non-fatal and continues with diagnostics and capture, although captured frames cannot be saved.

`wifi_init_softap()`

Wi-Fi initialization happens in several layers:

1. `nvs_flash_init()` opens non-volatile storage used by the Wi-Fi driver. If its pages are exhausted or from an incompatible NVS version, the code erases NVS and tries again. Erasing NVS also removes other values stored there.
2. `esp_netif_init()` initializes the TCP/IP network-interface layer.
3. `esp_event_loop_create_default()` creates the default event loop used by networking components.
4. `esp_netif_create_default_wifi_ap()` creates the AP interface and default DHCP server configuration.
5. `esp_wifi_init()` creates the Wi-Fi driver with ESP-IDF defaults.
6. A zero-initialized `wifi_config_t` receives the SSID, password, channel, and maximum client count. An empty password selects an open network; otherwise the code uses WPA2-PSK.
7. `esp_wifi_set_mode()`, `esp_wifi_set_config()`, and `esp_wifi_start()` enable AP mode.

Most of these calls use `ESP_ERROR_CHECK`. Unlike the SD path, an unexpected Wi-Fi error normally aborts the application instead of returning gracefully. After startup, the code reads and prints the AP address byte by byte. The default address is normally `192.168.4.1`.

`gpio_level_check()`

This is an electrical sanity check performed before any SCCB transactions:

1. `gpio_reset_pin()` clears any earlier configuration on SDA and SCL.
2. Both pins become inputs and their unassisted levels are logged.
3. Internal pull-ups are enabled, the task waits 10 ms, and both pins are read again.
4. Pull-ups are removed so the following SCCB driver can configure the pins.

Typical interpretations are:

Initial	With pull-up	Likely interpretation
High	High	Bus is idle or has external pull-ups
Low	High	Line was floating and the internal pull-up raised it
Low	Low	Line may be shorted, driven low, or attached to an unpowered device

High levels only prove that the lines are not stuck low; they do not prove that the sensor is powered, clocked, or able to acknowledge commands.

`sccb_debug_scan()`

The scan initializes SCCB on GPIO26/27 and probes valid 7-bit addresses from `0x03` through `0x77`. Failed probes are logged at debug level, so they may be hidden at the normal info log level. A successful acknowledgement is logged at info level.

For each responding address, the code reads registers `0x300A` and `0x300B` and combines them as `(high << 8) | low`. Those are common PID registers on some OmniVision sensors, but they are not universal for every camera. It also dumps `0x3000` through `0x3005` for additional evidence.

Finally, `SCCB_Deinit()` is essential: the diagnostic code must release the I2C controller before `esp_camera_init()` tries to acquire and configure the same bus. A detected address is evidence of communication; sensor identity should only be accepted when register reads also complete successfully.

`camera_init()`

`esp_camera_init(&camera_config)` performs much more than its wrapper suggests: it generates XCLK, configures the parallel camera peripheral and DMA, initializes SCCB, searches for a supported sensor, programs that sensor, and allocates the framebuffer. Any failure is logged and returned to `app_main()`.

After success, `esp_camera_sensor_get()` returns a `sensor_t` containing the detected ID and function pointers for sensor-specific controls. The code logs the PID and calls those function pointers to set neutral brightness, contrast, and saturation, then enables automatic white balance and exposure.

The five setting calls currently have no return-value checks. A log saying `Camera initialised successfully` confirms the main initialization, but does not prove every optional setting write succeeded.

`capture_frame()`

`esp_camera_fb_get()` waits for a completed frame and returns a `camera_fb_t *`. That structure owns the buffer pointer, byte length, width, height, and pixel format. A `NULL` result means capture failed, and the function returns without attempting file I/O.

For a valid frame, the function:

1. Logs size, dimensions, and format.
2. Checks that at least three bytes exist.
3. Compares the first bytes with the JPEG start signature `FF D8 FF`.
4. Builds `/sdcard/frame_NNNN.jpg` using a zero-padded frame number.
5. Opens the path in binary write mode, writes exactly `fb->len` bytes, closes the file, and reports a complete or partial write.
6. Calls `esp_camera_fb_return(fb)` in every path after successful acquisition.

Returning the framebuffer is mandatory. With only one buffer, forgetting that call would prevent later captures. A bad JPEG header produces a warning but is still written to the card, which preserves the data for diagnosis.

`app_main()` and complete control flow

The entry point coordinates the application:

```
boot
  -> log initial free heap
  -> optionally start SoftAP
  -> optionally mount SD card
  -> inspect SDA/SCL levels
  -> scan SCCB and release the bus
  -> initialize camera
       -> on failure: log and return
  -> log remaining heap
```

```
    -> capture frames 0..9, waiting 2 seconds after each attempt
    -> unmount SD card if mounted
    -> return from app_main
```

Returning after camera initialization fails ends the application task, but it does not power off or reboot the ESP32. Also, that early return currently skips SD unmounting when the card was mounted. On the successful path, `esp_vfs_fat_sdcard_unmount()` flushes and detaches the filesystem after the ten capture attempts.

The final `10 frames captured` message describes loop iterations, not verified successful images: `capture_frame()` returns `void`, so `app_main()` does not count failed captures or failed SD writes separately.

Troubleshooting

- **Camera initialization fails:** verify the ribbon cable orientation and pin mapping, then inspect the SCCB scan. Idle SDA and SCL should normally read high. External pull-ups in the 2.2-10 kOhm range may be required.
- **No SCCB device responds:** check 3.3 V, ground, XCLK, SDA, SCL, and the camera ribbon. A line that remains low can indicate a short or a sensor held in reset or power-down.
- **SD mount fails:** use a FAT-formatted card and confirm the board's SD wiring. Set `ENABLE_SD` to `0` to isolate camera and SDMMC pin conflicts.
- **Capture fails or memory is low:** keep `fb_count = 1`, use `CAMERA_FB_IN_DRAM`, or reduce `frame_size` to `FRAMESIZE_QQVGA`. PSRAM is disabled in the checked-in SDK configuration.
- **Flashing cannot connect:** hold GPIO0 low, reset the ESP32, and retry with a stable 5 V supply and crossed TX/RX connections.

Project layout

- `main/main.c` — application configuration, diagnostics, capture, Wi-Fi, and SD logic.
- `main/CMakeLists.txt` — application component dependencies.
- `components/esp32-camera/` — bundled camera driver and sensor support.
- `sdkconfig` — current ESP-IDF project configuration.
- `platformio.ini` — PlatformIO ESP-IDF environment.